

File ⁽¹⁾	Creada
Proyecto intermodular	5:01 p. m. - febrero 17, 2026

Status: #alto

Tags: ASIR

Proyecto intermodular

Repositorio del proyecto

El repositorio actualizado (incluido el servicio de Redis), está aquí: https://github.com/raul-profesor/Proy_intermodular_ASIR_25-26

Fases

1. Desplegar 6 instancias Debian (4 para Nginx, 1 para MySQL y 1 para Redis)
 1. En el mismo tiempo de creación de instancias, en el campo **User Data**, meter un script en bash para instalar Docker y sus componentes (<https://docs.docker.com/engine/install/debian/#install-using-the-repository>), realizar los pasos post instalación (<https://docs.docker.com/engine/install/linux-postinstall>), así como instalar el paquete **nfs-common** para poder montar posteriormente el EFS
2. Crear y aplicar los grupos de seguridad
3. Crear clúster
4. Etiquetar nodos
5. Unir nodos al clúster
6. Crear cuenta en Docker Hub
7. Build de la imagen de php-fpm y subirla al Docker Hub
8. Montar el EFS en todas las máquinas menos en la de Redis y copiar allí los archivos necesarios
9. Desplegar el stack
10. Reconstruir el build de php-fpm para incluir el plugin de Redis (en el caso de no haberlo hecho en el inicio)
11. Modificar el **docker-compose.db.yaml** para incluir el servicio de Redis (si no se ha hecho en el inicio)
12. Redespregar (si se ha añadido Redis con posterioridad)
13. Balanceo de carga (ALB)
 1. Grupo de seguridad que permita el tráfico 443 (HTTPS)
14. Registrar dominio gratuito
15. Solicitar certificado para HTTPS en el servicio ACM de AWS
16. Incluir en nuestro DNS el CNAME que nos indique ACM

Creación de instancias

Instancias (6) Información Conectar Estado de la instancia Acciones Lanzar					
Buscar instancia por atributo o etiqueta (case-sensitive) Todos los estados					
Estado de la instancia = running Quitar los filtros					
☐	Name	ID de la instancia	Estado de la i...	Tipo de inst...	
☐	debian-redis	i-0a6aff0ab9ff38728	✓ En ejecución	t3.micro	
☐	Debian-cluster-1	i-07b70ea60a90798d9	✓ En ejecución	t2.small	
☐	Debian-cluster-2	i-0069ae08a4b778002	✓ En ejecución	t2.small	
☐	Debian-cluster-3	i-09976e8e608b324bb	✓ En ejecución	t2.small	
☐	Debian-cluster-4	i-020bfb725b08fe641	✓ En ejecución	t2.small	
☐	Debian-cluster-mysql	i-0eb56f4177c6ce172	✓ En ejecución	t2.small	

Grupos de seguridad

Necesitaremos crear un grupo de seguridad que aplicaremos todos los nodos del clúster.

Los puertos necesarios en las reglas de entrada se indican en la tabla y sirven para que los nodos puedan comunicarse entre ellos a través de la red de **overlay** del clúster, así como para que el **manager** pueda controlar a los **workers**:

	Name	ID de la regla del grupo de seguridad	Versión de IP	Tipo	Protocolo	Intervalo de puertos	Origen	Descripción
-		sgr-0051228785462f367	IPv4	SSH	TCP	22	0.0.0.0/0	-
-		sgr-00a9405f7659040cd	-	TCP personalizado	TCP	2377	sg:0d2e4eefed2c362b3 / SG-SWARM	manager swarm control plane
-		sgr-0673effbc05132631	-	TCP personalizado	TCP	7946	sg:0d2e4eefed2c362b3 / SG-SWARM	overlay network
-		sgr-0cf36265dece76c0d	-	UDP personalizado	UDP	7946	sg:0d2e4eefed2c362b3 / SG-SWARM	overlay network
-		sgr-0dde24c722b901e03	-	UDP personalizado	UDP	4789	sg:0d2e4eefed2c362b3 / SG-SWARM	overlay network

Observad que para mayor seguridad, el único tráfico de entrada que se permite a estos nodos tiene como origen el propio grupo de seguridad. Puesto que este grupo sólo se aplica a los nodos del clúster, en la práctica le estamos diciendo que sólo permita este tráfico entre ellos y nadie más.

Creación del clúster

En el nodo que será manager — inicializar Swarm (reemplaza <MANAGER_IP> por la IP accesible a otros nodos):

```
docker swarm init --advertise-addr <MANAGER_IP>
```

SHELL

Obtener el token necesario para unir workers (en el manager) y ejecútalo en cada nodo worker:

```
docker swarm join-token worker
#Salida ejemplo: docker swarm join --token SWMTKN-1-... <MANAGER_IP>:2377
#Ejecutar la línea completa en cada worker
```

Verificar en el manager:

```
docker node ls
```

Etiquetar nodos para saber dónde se van a desplegar los servicios posteriormente. Desde el manager:

```
docker node update --label-add web=true web
docker node update --label-add web=true web
docker node update --label-add web=true web
docker node update --label-add web=true web
docker node update --label-add db=true db
docker node update --label-add db=true redis
```

Crear la red overlay compartida (en el manager):

```
docker network create -d overlay app-network
```

Creación de cuenta den Docker Hub y build de imagen

Deberéis crearos una cuenta en <https://hub.docker.com/>.

Tras ello, tendréis que crearos la imagen que Docker Swarm utilizará para php-fpm puesto que en el momento del despliegue Docker Swarm no es capaz de crear imágenes a partir de un Dockerfile como sí lo hacía docker compose.

```
docker login -u tu_usuario

docker build -t webstack-php:latest -f ./dockerfiles/Dockerfile.web ./dockerfiles

docker tag webstack-php:latest tu_usuario/webstack-php:latest

docker push tu_usuario/webstack-php:latest
```

Montaje del EFS

Esto ya lo hemos hecho con anterioridad así que no necesita mucha explicación. Se ha de montar en todos los nodos menos en el de Redis, que no va a hacer uso del almacenamiento compartido.

No copiéis directamente, ya conocéis el proceso para obtener el nombre DNS del EFS y que AWS os dé el comando exacto que debéis introducir.

Importante cerciorarse de que se ha instalado correctamente *nfs-common*.

```
# Verificar si /mnt/efs está montado
mount | grep efs

# Si no, montarlo
sudo mkdir -p /mnt/efs
sudo mount -t nfs4 -o
nfsvers=4.1,rsize=1048576,wsiz=1048576,hard,timeo=600,retrans=2,noresvport fs-
03c6346f07295d19e.efs.us-east-1.amazonaws.com:/ /mnt/efs

# Para que sea persistente, añadir a /etc/fstab:
echo "fs-03c6346f07295d19e.efs.us-east-1.amazonaws.com:/ /mnt/efs nfs4
nfsvers=4.1,rsize=1048576,wsiz=1048576,hard,timeo=600,retrans=2,noresvport,_netd
ev 0 0" | sudo tee -a /etc/fstab
```

Tras esto es necesario crear en el EFS las carpetas correspondientes a los volúmenes que utilizan los `docker-compose.*.yaml` y copiar en cada carpeta los archivos correspondientes.

Despliegue del stack en el clúster

Para que pueda hacer uso de las variables de entorno definidas en `.env`:

```
export $(cat .env | grep -v "^#" | xargs)
```

Desplegar la base de datos en su stack (en el manager):

```
docker stack deploy -c db-stack.yml dbstack
```

Desplegar la stack web (en el manager):

```
docker stack deploy -c docker-compose.yml webstack
```

SHELL

Comprobaciones y diagnóstico:

```
# Listar stacks y servicios
docker stack ls
docker stack services dbstack
docker stack services webstack

# Ver tareas de un servicio
docker service ps webstack_nginx
docker service ps webstack_php
docker service ps dbstack_mysql
docker service ps dbstack_redis

# Ver logs de un servicio
docker service logs --follow webstack_php
```

Balanceo de carga

De la misma forma, ya hemos visto como crear un ALB:

- Debéis crear un target group que incluya los 4 nodos *nginx* puesto que es el único tráfico que se balanceará (no así el de MySQL ni el de Redis)

Destinos
Monitorización
Comprobaciones de estado
Atributos
Etiquetas

Destinos registrados (4) Info

Mitigación de anomalías: **No aplicable**
Anular el registro
Registrar destinos

Los grupos de destinos enrutan las solicitudes a destinos individuales registrados mediante el protocolo y el número de puerto que especifique. Las comprobaciones de estado se realizan en todos los destinos registrados de acuerdo con la configuración de comprobación de estado del grupo de destinos. La detección de anomalías se aplica automáticamente a los grupos de destinos con HTTP/HTTPS con al menos 3 destinos en buen estado.

1

☐	ID de instancia	Nombre	Puerto	Zona	Estado
☐	i-09976e8e608b324bb	Debian-cluster-3	80	us-east-1c (use...	Healthy
☐	i-020bfb725b08fe641	Debian-cluster-4	80	us-east-1c (use...	Healthy
☐	i-07b70ea60a90798d9	Debian-cluster-1	80	us-east-1c (use...	Healthy
☐	i-0069ae08a4b778002	Debian-cluster-2	80	us-east-1c (use...	Healthy

- Debéis crear un grupo de seguridad que aplicaréis al balanceador de carga:
 - Debe permitir el tráfico de entrada al puerto 443 (HTTPS) desde cualquier origen puesto que estará expuesto a Internet
 - Debe permitir el tráfico hacia al puerto 80 (HTTP) y con destino el grupo de seguridad de los nodos del clúster
 - De la misma forma se debe añadir una regla de entrada en el grupo de seguridad de los nodos del clúster que permita el tráfico de entrada en el puerto 80 al tráfico con origen el grupo de seguridad del ALB.

Esto es así porque el balanceador será el encargado de manejar el certificado para HTTPS, será el que descifrará el tráfico y lo enviará a los nodos, por lo que el tráfico desde el exterior al ALB será HTTPS y, cuando éste lo descifre y lo reenvíe, ya será sin descifrar al puerto 80 de los nodos NGINX.

Registrar dominio gratuito

Este proceso también lo conocéis y probablemente lo tendréis hecho:

- Debéis registrar un dominio gratuito.
- Tras ello, debéis crear un alias o CNAME donde hacéis corresponder un CNAME al nombre DNS de vuestro ALB (podéis verlo sin problemas en AWS)

De esta forma, cada vez que accedáis a vuestro nombre alias, estaréis accediendo al ALB.

Certificado HTTPS

Si todo ha ido bien, usando nuestro CNAME en el navegador accederemos al ALB y de ahí a nuestra aplicación.

Esto no es del todo cierto, puesto que el ALB sólo nos permite el puerto 443 pero no tenemos ningún certificado configurado todavía. Para ello, una forma de solucionarlo es hacer uso del servicio ACM de AWS.

Preguntar a Amazon Q

ID de cuenta: 6374...
voclabs/user44172...

Servicios
Ver los 15 resultados

Servicios (15)
Características (14)
Publicaciones de blog (9)
Documentación (945)
Artículos de conocimiento (51)
Modelos (118)

AWS Private Certificate Authority
Servicio de la entidad de certificación privada administrada

Características principales
Conector de entidad de certificación privada (CA) de AWS para Active Directory
Conector de AWS Private CA para SCEP

Una vez allí, debéis solicitar un certificado de tipo público. Recordad el dominio (CNAME) para el que estáis pidiendo el certificado.

No hace falta que cambiéis ninguna opción puesto que vamos a validar el certificado por DNS.

Cuando lo hayáis emitido estará pendiente de validación y además, con el fin de verificar que el certificado lo estáis pidiendo para un dominio vuestro y tenéis el control del mismo, os pide que añadáis un registro en él. En concreto os pide que creéis un CNAME y un valor para el CNAME.

Tan fácil como ir a la página donde hemos configurado anteriormente nuestro dominio gratuito y meter ambos valores donde corresponde.

Al cabo de unos minutos el certificado estará validado y podremos acceder sin problemas a nuestra aplicación.